

CSC241 – Object-Oriented Programming

Ahmad Talal

SP26-BAI-008

BAI Section SP_2026

Sir Shahid Bhatti

Setup

Working folder created and toolchain verified from Command Prompt before starting the tasks.

```
cd C:\CSC241\CIIT-SP26-BAI-008-LHR
mkdir Lab02
cd Lab02
java -version
javac -version
git --version
```

Task 1 – Build a Class and Prove Independent Object State

predictions.txt (before running):

Task 1 prediction:
All three students should print with the values I assigned.
After changing only s2's completedCredits, s1 and s3 should stay the same because they are separate objects in memory.

Student.java

```
class Student {
    String studentId;
    String name;
    int completedCredits;
}
```

Task1Demo.java

```
public class Task1Demo {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.studentId = "SP26-BAI-007";
        s1.name = "Abeer Amina";
        s1.completedCredits = 15;

        Student s2 = new Student();
        s2.studentId = "SP26-BAI-011";
        s2.name = "Ali Ishtiaq";
        s2.completedCredits = 18;

        Student s3 = new Student();
        s3.studentId = "SP26-BAI-023";
        s3.name = "Abdul Rehman Azam";
        s3.completedCredits = 12;

        System.out.println("Before change:");
        System.out.println(s1.studentId + " " + s1.name + " " + s1.completedCredits);
        System.out.println(s2.studentId + " " + s2.name + " " + s2.completedCredits);
        System.out.println(s3.studentId + " " + s3.name + " " + s3.completedCredits);
    }
}
```

```

        s2.completedCredits = 21;

        System.out.println();
        System.out.println("After changing s2 only:");
        System.out.println(s1.studentId + " " + s1.name + " " + s1.completedCredits);
        System.out.println(s2.studentId + " " + s2.name + " " + s2.completedCredits);
        System.out.println(s3.studentId + " " + s3.name + " " + s3.completedCredits);

        // s1 and s3 are separate objects, each with its own memory space,
        // so changing a field on s2 has no effect on s1 or s3.
    }
}

```

Console output:

Task 1 Concept Check

1. Difference between the class and s1/s2/s3: Student is the blueprint that defines what fields every student has, while s1, s2, and s3 are actual objects built from that blueprint, each with its own memory and its own values.

2. Why changing s2.completedCredits doesn't affect s1 or s3: Each object created with new Student() gets its own separate copy of the instance variables, so a change made through one reference (s2) only updates that object's memory, not the others.

3. Which fields are instance variables and why: studentId, name, and completedCredits are all instance variables because they represent state that belongs to a specific student and must stay alive for as long as that object exists, unlike a local variable which disappears once a method ends.

Task 2 – Add Behavior: Parameters, Local Variables, and Return Values

predictions.txt (before running):

Task 2 prediction:
 addCredits should increase completedCredits by the argument I pass.
 remainingCredits should return degreeCredits minus completedCredits.
 summary should print one line with id, name and credits.

Student.java (revised)

```

class Student {
    String studentId;
    String name;
    int completedCredits;

    void addCredits(int credits) {
        completedCredits = completedCredits + credits;
    }

    int remainingCredits(int degreeCredits) {
        int remaining = degreeCredits - completedCredits;
        return remaining;
    }

    String summary() {
        return studentId + " - " + name + " - " + completedCredits + " credits completed";
    }
}

```

```
}  
}
```

Task2Demo.java

```
public class Task2Demo {  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        s1.studentId = "SP26-BAI-007";  
        s1.name = "Abeer Amina";  
        s1.completedCredits = 15;  
  
        Student s2 = new Student();  
        s2.studentId = "SP26-BAI-011";  
        s2.name = "Ali Ishtiaq";  
        s2.completedCredits = 18;  
  
        s1.addCredits(3);  
        s2.addCredits(6);  
  
        int remaining1 = s1.remainingCredits(130);  
        int remaining2 = s2.remainingCredits(130);  
  
        System.out.println(s1.summary());  
        System.out.println("Remaining credits for s1: " + remaining1);  
  
        System.out.println(s2.summary());  
        System.out.println("Remaining credits for s2: " + remaining2);  
    }  
}
```

Console output:

```
C:\CSC241\CIIT-SP26-BAI-008-LHR\Lab02>javac Student.java Task2Demo.java  
C:\CSC241\CIIT-SP26-BAI-008-LHR\Lab02>java Task2Demo  
SP26-BAI-008 - Abeer Amina - 18 credits completed  
Remaining credits for s1: 112  
SP26-BAI-011 - Ali Ishtiaq - 24 credits completed  
Remaining credits for s2: 106  
  
C:\CSC241\CIIT-SP26-BAI-008-LHR\Lab02>  
```[cite: 6]
```

### Task 2 Concept Check

**1. credits (parameter) vs completedCredits (field):** credits is a temporary value handed in when addCredits is called and only exists during that call, while completedCredits is the object's own field that keeps its value between method calls.

**2. Why remaining should be local, not a field:** remaining is only needed to hold a calculation while the method runs and has no meaning once the method returns, so making it a field would just waste memory and could confuse it with real student state.

**3. What the return type tells the caller:** int as the return type tells the caller that calling remainingCredits(...) will hand back a whole number it can store, print, or use in further calculations.

## Task 3 – Method Signatures and Overloading

*predictions.txt (before running):*

```
Task 3 prediction:
enroll("CSC241") -> Enrolled in CSC241
enroll("CSC241", 2) -> Enrolled in CSC241 section 2
enroll(241) -> Enrolled using numeric code 241
enroll() and enroll("241","2") should not compile - no matching signature.
```

### OverloadDemo.java

```
public class OverloadDemo {

 // enroll(String) : (String)
 static void enroll(String courseCode) {
 System.out.println("Enrolled in " + courseCode);
 }

 // enroll(String, int) : (String, int)
 static void enroll(String courseCode, int section) {
 System.out.println("Enrolled in " + courseCode + " section " + section);
 }

 // enroll(int) : (int)
 static void enroll(int numericCourseCode) {
 System.out.println("Enrolled using numeric code " + numericCourseCode);
 }

 public static void main(String[] args) {
 enroll("CSC241");
 enroll("CSC241", 2);
 enroll(241);

 // enroll(); // invalid - no method takes zero arguments
 // enroll("241", "2"); // invalid - no overload takes (String, String)
 }
}
```

*Console output (valid calls):*

```
C:\CSC241\CIIT-SP26-BAI-008-LHR\Lab02>javac Student.java Task1Demo.java
C:\CSC241\CIIT-SP26-BAI-008-LHR\Lab02>java Task1Demo
Before change:
SP26-BAI-008 Abeer Amina 15
SP26-BAI-011 Ali Ishtiaq 18
SP26-BAI-023 Abdul Rehman Azam 12

After changing s2 only:
SP26-BAI-008 Abeer Amina 15
SP26-BAI-011 Ali Ishtiaq 21
SP26-BAI-023 Abdul Rehman Azam 12

C:\CSC241\CIIT-SP26-BAI-008-LHR\Lab02>
```[cite: 7]
```

```
C:\CSC241\CIIT-SP26-BAI-008-LHR\Lab02>javac OverloadDemo.java
C:\CSC241\CIIT-SP26-BAI-008-LHR\Lab02>java OverloadDemo
Enrolled in CSC241
Enrolled in CSC241 section 2
Enrolled using numeric code 241

C:\CSC241\CIIT-SP26-BAI-008-LHR\Lab02>
```[cite: 5]
```

*Invalid overload attempt (differs only by return type) and the compiler message captured in compiler\_notes.txt:*

```
// Added temporarily to test an invalid overload:
// int enroll(String courseCode) {
// System.out.println("Enrolled in " + courseCode);
// return 1;
// }
```

```
C:\CSC241\CIIT-SP26-BAI-008-LHR\Lab02>REM invalid overload attempt - differs on
OverloadDemo.java:9: error: method enroll(String) is
already defined in class OverloadDemo
 static int enroll(String courseCode) {
 ^
1 error

C:\CSC241\CIIT-SP26-BAI-008-LHR\Lab02>
```

### Task 3 Concept Check

**1. Signature of enroll(String courseCode, int section):** enroll(String, int) — the method name plus the ordered list of parameter types.

**2. Why String/int overloads work but return-type-only doesn't:** enroll(String) and enroll(int) are valid overloads because their parameter types are different, but Java's overload resolution only looks at the method name and parameter list, not the return type, so two methods that differ only in what they return look identical to the compiler and are rejected.

**3. Which overload enroll("CSC241", 2) selects:** The compiler picks enroll(String, int) because it matches the call's argument types and count exactly, and Java resolves overloads at compile time based on that match.

## Task 4 – Prove Java Pass-by-Value

*predictions.txt (before running):*

Task 4 prediction:

A: number stays 10 after changeNumber - the argument value is copied into the parameter.

B: s1.completedCredits becomes 99 - changeStudent mutates the object the reference points to.

C: s2.name stays the same - replaceStudent only redirects the local parameter, not the caller's variable.

### PassByValueDemo.java

```
public class PassByValueDemo {

 static void changeNumber(int x) {
 x = 99;
 }

 static void changeStudent(Student st) {
 st.completedCredits = 99;
 }

 static void replaceStudent(Student st) {
 st = new Student();
 st.name = "Temporary";
 }

 public static void main(String[] args) {
 // Experiment A
 int number = 10;
 System.out.println("A - Before: " + number);
 changeNumber(number);
 System.out.println("A - After: " + number);

 // Experiment B
 Student s1 = new Student();
 s1.name = "Ali Ishtiaq";
 s1.completedCredits = 18;
 System.out.println();
 System.out.println("B - Before: " + s1.completedCredits);
 changeStudent(s1);
 System.out.println("B - After: " + s1.completedCredits);

 // Experiment C
 Student s2 = new Student();
 s2.name = "Abeer Amina";
 System.out.println();
 System.out.println("C - Before: " + s2.name);
 replaceStudent(s2);
 }
}
```

```
 System.out.println("C - After: " + s2.name);
 }
}
```

Console output:

```
C:\CSC241\CIIT-SP26-BAI-008-LHR\Lab02>javac Student.java PassByValueDemo.java
C:\CSC241\CIIT-SP26-BAI-008-LHR\Lab02>java PassByValueDemo
A - Before: 10
A - After: 10

B - Before: 18
B - After: 99

C - Before: Abeer Amina
C - After: Abeer Amina

C:\CSC241\CIIT-SP26-BAI-008-LHR\Lab02>
```[cite: 8]
```

Required explanation:

changeStudent(...) can affect the object because the argument value is copied into the parameter, and that copied value is a reference pointing to the same Student object as s1 — so st.completedCredits = 99 changes the one real object both variables point to. replaceStudent(...) cannot redirect the caller's variable because st = new Student() only changes what the local parameter st points to; s2 in main still holds its original reference and never finds out about the reassignment.

Task 4 Concept Check

- 1. Why number keeps its old value:** changeNumber(int x) copies number's value into x, so reassigning x = 99 only changes that local copy — the argument value is copied into the parameter, and number in main is untouched.
- 2. Why the mutation through st is visible to the caller:** The reference stored in s1 is copied into st, but both the original and the copy point to the same Student object in memory, so changing a field through st changes the same object the caller sees.
- 3. Why st = new Student() doesn't redirect the caller's reference:** Because st is a local copy of the reference, reassigning st only makes that local copy point elsewhere — it cannot reach back and change what variable s2 in main points to.

Task 5 – Integrated BAI Challenge: AIExperiment

predictions.txt (before running):

```
Task 5 prediction:
Each AIExperiment should keep its own completedEpochs/targetEpochs.
runEpochs should increase completedEpochs; the overload should add both amounts.
remainingEpochs should return targetEpochs - completedEpochs.
Passing exp1 into a helper method should still update exp1's real completedEpochs.
```

AIExperiment.java

```
class AIExperiment {
```

```

String experimentName;
int completedEpochs;
int targetEpochs;

void runEpochs(int epochs) {
    completedEpochs = completedEpochs + epochs;
}

void runEpochs(int epochs, int bonusEpochs) {
    completedEpochs = completedEpochs + epochs + bonusEpochs;
}

int remainingEpochs() {
    int remaining = targetEpochs - completedEpochs;
    return remaining;
}

String status() {
    return experimentName + ": " + completedEpochs + "/" + targetEpochs + " epochs completed";
}
}

```

AIExperimentDemo.java

```

public class AIExperimentDemo {

    // helper method - receives a reference and mutates the real object
    static void addBonusRun(AIExperiment exp) {
        exp.runEpochs(5);
    }

    public static void main(String[] args) {
        AIExperiment exp1 = new AIExperiment();
        exp1.experimentName = "Image Classifier";
        exp1.completedEpochs = 10;
        exp1.targetEpochs = 50;

        AIExperiment exp2 = new AIExperiment();
        exp2.experimentName = "Chatbot Fine-tune";
        exp2.completedEpochs = 5;
        exp2.targetEpochs = 30;

        System.out.println("Initial state:");
        System.out.println(exp1.status());
        System.out.println(exp2.status());

        exp1.runEpochs(8);
        exp2.runEpochs(4, 2);

        System.out.println();
        System.out.println("After running epochs:");
        System.out.println(exp1.status());
        System.out.println(exp2.status());

        System.out.println();
        System.out.println("Remaining epochs:");
        System.out.println(exp1.experimentName + " remaining: " + exp1.remainingEpochs());
        System.out.println(exp2.experimentName + " remaining: " + exp2.remainingEpochs());

        addBonusRun(exp1);
    }
}

```

```
        System.out.println();
        System.out.println("After helper method bonus run on exp1:");
        System.out.println(exp1.status());
    }
}
```

Console output:

```
C:\CSC241\CIIT-SP26-BAI-008-LHR\Lab02>javac AIExperiment.java AIExperimentDemo.java
C:\CSC241\CIIT-SP26-BAI-008-LHR\Lab02>java AIExperimentDemo
Initial state:
Image Classifier: 10/50 epochs completed
Chatbot Fine-tune: 5/30 epochs completed

After running epochs:
Image Classifier: 18/50 epochs completed
Chatbot Fine-tune: 11/30 epochs completed

Remaining epochs:
Image Classifier remaining: 32
Chatbot Fine-tune remaining: 19

After helper method bonus run on exp1:
Image Classifier: 23/50 epochs completed

C:\CSC241\CIIT-SP26-BAI-008-LHR\Lab02>
```[cite: 9]
```

## Task 5 Concept Check

- 1. Instance variables vs. local calculations:** `experimentName`, `completedEpochs`, and `targetEpochs` are instance variables because they must persist as long as the experiment exists, while a value like `remaining` in `remainingEpochs()` is only needed during that one calculation, so it stays local.
- 2. A state-changing method vs. an information-only method:** `runEpochs(int epochs)` changes object state by updating `completedEpochs`, while `remainingEpochs()` only computes and returns a value without modifying any field.
- 3. Overloaded signatures used:** `runEpochs(int)` and `runEpochs(int, int)` — the compiler can tell them apart because they take a different number of parameters, even though the method name is the same.

## GitHub Checkpoint

```
git status
git add Lab02
git commit -m "CSC241 Lab02 tasks 1-4"
git add Lab02
git commit -m "Complete Lab02 integrated AIExperiment task"
git push
git status
```

## Five-Minute Exit Check

**1. Class vs. object:** A class is the blueprint describing what fields and methods its objects will have; an object is an actual instance created from that blueprint with its own memory and values.

**2. Why s1.completedCredits stays unaffected by a change to s2:** s1 and s2 are separate objects, each holding its own independent copy of completedCredits, so a change made through one does not touch the other.

**3. Signature of enroll(String, int):** enroll(String, int).

**4. Why return type alone can't distinguish overloads:** Java resolves which overload to call at compile time using only the method name and parameter list, so two methods differing only in return type look like duplicate declarations and the compiler rejects them.

**5. Mutating through a copied reference vs. reassigning it:** The copied reference still points to the same object in memory, so changing a field through it is visible everywhere, but reassigning the copied reference itself only redirects that local copy and leaves the caller's original variable pointing at the original object.